

Universidad Nacional de La Matanza
Departamento de Ingeniería e Investigaciones Tecnológicas

Sistemas Operativos Avanzados

Métricas e IA

Integrantes:

- Passarelli Agustin Ezequiel
- Muñoz Palazzo Ezequiel
- Mercaich Ignacio Gabriel
- Mola Claudio Tomás

Fecha de entrega: 06/07/2026

Herramienta de IA utilizada: Github Copilot

Modelo de IA utilizado: Raptor Mini & Gemini Pro

Prompt técnico utilizado:

LENGUAJE DE PROGRAMACIÓN: Java
PLATAFORMA: Desarrollar una app de Android con Internet de las cosas (IoT) que se conecte con MQTT a una serie de tópicos y envíe mensajes por Telegram.
FUNCIONALIDAD A IMPLEMENTAR: El sistema actúa como un middleware o pasarela (Gateway) de asistencia para emergencias ("Nonófono"). Su función principal es recibir peticiones o alertas físicas provenientes de un microcontrolador (ESP32) vía MQTT, procesar dichos eventos de manera asíncrona, descargar audios vía HTTP cuando corresponda, y retransmitir las notificaciones y archivos multimedia a una lista de contactos familiares utilizando la API de un bot de Telegram. A su vez, permite localizar el dispositivo físico enviando comandos desde el celular mediante la detección de movimientos bruscos.
REQUISITOS ESPECIFICOS Y ARQUITECTURA:

- Arquitectura de Datos y Gestor: Toda la lógica de negocio y persistencia de datos debe estar rigurosamente encapsulada en una clase Gestor (ContactosManager).
    - ESTRICTO: Debes crear una clase POJO Contacto.java con los atributos String nombre, String telefono, y long chatId. El ContactosManager debe serializar y deserializar obligatoriamente un ArrayList<Contacto> usando SharedPreferences. Bajo ningún concepto utilices una simple lista de Strings.
- Actividades (Activities) y XML:
    - MainActivity (activity_main.xml): Panel de control central. Debe contener botones de navegación hacia las otras pantallas y etiquetas de texto (TextView) para monitorear visualmente el estado de conexión de Telegram y MQTT en tiempo real. Se encarga de lanzar el servicio de red.
    - AgregarContactoActivity: Módulo para gestionar el directorio. Debe incluir un formulario con validación por expresiones regulares (entre 8 y 15 caracteres) y un RecyclerView (con su respectivo Adapter) para listar, añadir (hasta un máximo de 10) y eliminar contactos.
    - LocalizarActivity: Pantalla de rastreo. Debe implementar de forma obligatoria el sensor del acelerómetro. Si el usuario agita el celular (shake) y la magnitud supera un umbral de 15.0f, se debe disparar un evento (con un cooldown de 6 segundos) que publique un mensaje MQTT para hacer sonar el hardware remoto.

- Integración con Telegram (Long Polling): La API de Telegram NO acepta números de teléfono para enviar mensajes, requiere un chat_id. Debes implementar un mecanismo de Long Polling hacia https://api.telegram.org/bot<TOKEN>/getUpdates (en un hilo secundario en MainActivity o en el Servicio) que: lea los mensajes entrantes, filtre los números de teléfono recibidos, los empareje con el directorio local de contactos, extraiga el chat_id y actualice el objeto Contacto guardándolo en el ContactosManager.
- Servicio en Segundo Plano (Foreground Service):
    - Implementar un Foreground Service (NonofonoService) con la bandera START_STICKY.
    - ESTRICTO: En el AndroidManifest.xml, el servicio debe estar declarado con android:foregroundServiceType="dataSync".
- Cliente MQTT:
    - ESTRICTO: Utiliza la librería estándar de Java org.eclipse.paho.client.mqttv3.MqttClient (o MqttAsyncClient). Instáncialo y manéjalo en un hilo secundario (new Thread) dentro del servicio. ESTÁ ESTRICTAMENTE PROHIBIDO usar MqttAndroidClient (org.eclipse.paho.android.service), ya que es una librería obsoleta que causará NoClassDefFoundError.
- Notificación Persistente: El servicio debe anclar un NotificationChannel informando que el "Nonófono" está activo. Debe incluir un PendingIntent con un botón explícito para detener el servicio y desconectar la red.
- Descarga de Audio HTTP: Crear la clase AudioDownloader que ejecute GETs (puerto 80) al ESP32 para descargar archivos .wav y los retransmita mediante POST a /sendAudio de Telegram en formato multipart/form-data.
    - ESTRICTO: Como el ESP32 transmite en HTTP puro, debes agregar android:usesCleartextTraffic="true" en la etiqueta <application> del AndroidManifest.xml para evitar el error CleartextNotPermittedException.

TÓPICOS MQTT:

- Suscripciones (Subscribe): nonofono/emergencias, nonofono/mensajes/audio, nonofono/mensajes/predeterminado, nonofono/live.
- Publicaciones (Publish): nonofono/encontrar, nonofono/mensajes/audio/ack, nonofono/mensajes/predeterminado/ack, nonofono/config/contactos.

TIPOS DE DATOS (JSON STRICT FORMAT): El código del ESP32 es inmutable. Debes parsear y serializar respetando esta estructura:

- Entrada (Desde ESP32):
    - Emergencias: {"evento": "emergencia", "contenido": {"mensaje": "Texto a enviar"}}
    - Audio: {"evento": "audio", "contenido": {"ip": "192.168.0.X", "endpoint": "/descargar_audio", "id_mensaje": 123, "destinatario": "1122334455"}}
    - Texto Rápido: {"evento": "mensaje-predefinido", "contenido": {"telefono": "1122334455", "contenido": "Mensaje de texto rápido"}}
    - Live: {"evento": "live", "contenido": {}}
- Salida (Hacia ESP32):

    - Contactos (Publicar 500ms tras evento live): [{"name": "Juan", "phone": "1122334455"}, {"name": "Maria", "phone": "1199887766"}]
    - Localizar: {"action":"localizar"}
    - ACKs: {"exito":"Ok"}

RESTRICCIONES:

1. Funcionar mínimo en Android 10 (API 29). Requiere permisos POST_NOTIFICATIONS y FOREGROUND_SERVICE_DATA_SYNC en el Manifest.
2. CONCURRENCIA (NetworkOnMainThreadException): Todas las operaciones de red (peticiones HTTP a Telegram o descargas de audio) lanzadas desde los callbacks de MQTT DEBEN envolverse en un hilo secundario (new Thread(() -> { ... }).start();).
3. La sincronización UI/Red debe ser no bloqueante usando BroadcastReceivers y runOnUiThread.
4. Token de Telegram a usar: 8322654023:AAHhtylDzV06H71m-Xko3mhbg2TQ3J5brdA
5. MQTT SSL Obligatorio (SSLSocketFactory.getDefault()):
    - Cluster: ssl://e32525b034cd4ef5a01f6e9715295f64.s1.eu.hivemq.cloud
    - User: admin_prueba / Pass: Nonofono8
6. Tolerancia a fallos: Timeouts estrictos en HTTP. Desconexión MQTT debe ejecutar client.close().
7. El proyecto ya está creado. No inicialices desde cero.

Cantidad de tokens de IA utilizados:

55 créditos ===► 0.55 dólares ===► 200.000 tokens de salida y 50.000 tokens de entrada aproximadamente.

Tiempo para obtener una versión funcional:

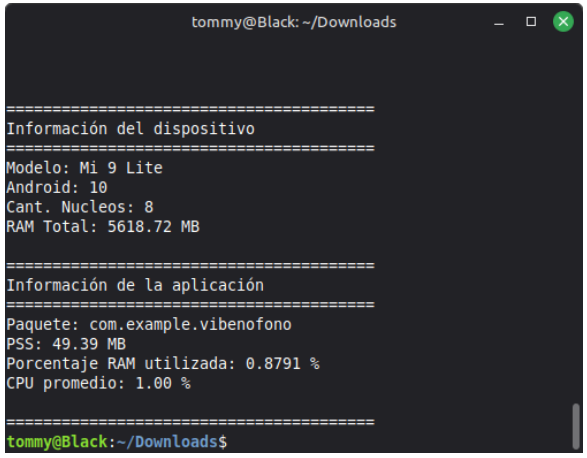
- **Manual: 30 días**
- **Vibe Coding: 4 hs**

Código en Git: https://github.com/UNLAM-SOA/2026-SOA-Q1-L3/tree/main/Android

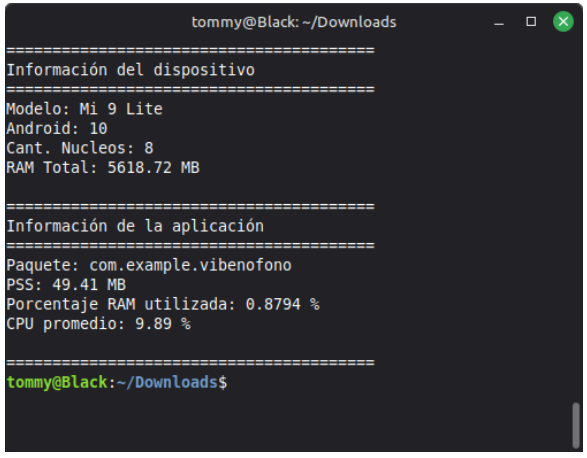
Métricas CPU y memoria

Proyecto con Vibecoding

Idle:

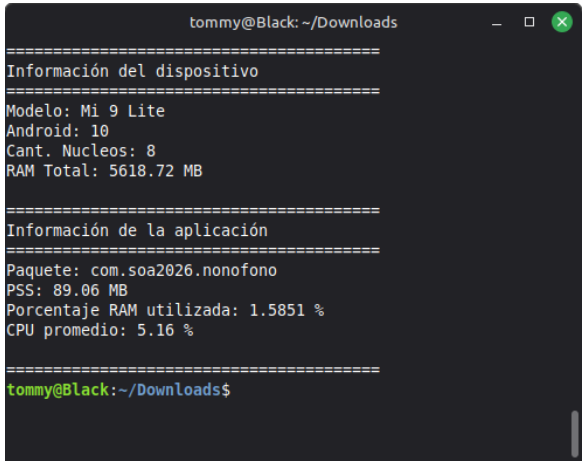


Shake:

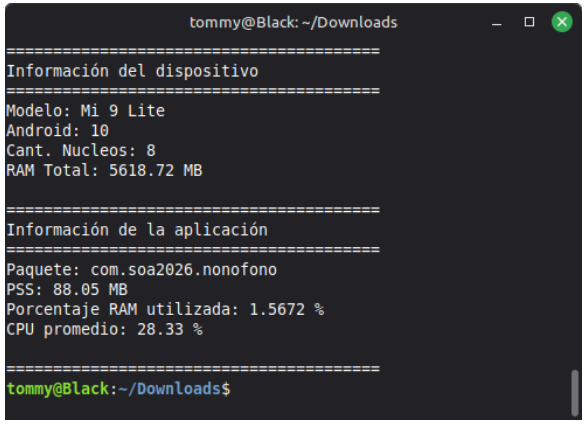


Proyecto sin Vibecoding

Idle:



Shake:



RÚBRICA FUNCIONALIDAD ANDROID (Vibecoding)

CRITERIOS	SUFICIENTEMENTE LOGRADO (A)	MEDIANAMENTE LOGRADO (B)	INSUFICIENTEMENTE LOGRADO (C)	Puntaje	Eval
FUNCIONA SIN ERRORES	Compila y funciona sin errores.	Funciona con errores menores o warnings.	Presenta errores que impiden su uso.	A: Hasta 20 pts. B: Hasta 10 pts. C: 0	10
UTILIZA EJECUCIÓN EN BACKGROUND	Implementa correctamente tareas en background.	Implementa background con algunas fallas.	No implementa tareas en background.	A: Hasta 10 pts. B: Hasta 5 pts. C: 0	8
UTILIZA SENSORES Y PERMISOS DE ANDROID	Utiliza correctamente sensores y permisos.	Presenta problemas menores en sensores o permisos.	Sensores o permisos no funcionan correctamente.	A: Hasta 20 pts. B: Hasta 10 pts. C: 0	20
COMUNICACIÓN CON EL EMBEBIDO	Comunicación estable en ambos sentidos.	Comunicación parcial o inestable.	La comunicación falla.	A: Hasta 20 pts. B: Hasta 10 pts. C: 0	10
CUMPLIMIENTO DEL OBJETIVO FUNCIONAL	Cumple todos los requisitos funcionales.	Cumple parcialmente los requisitos.	No cumple los requisitos mínimos.	A: Hasta 20 pts. B: Hasta 10 pts. C: 0	10
DISEÑO DE LA APP	Interfaz clara y fácil de usar.	Interfaz funcional con algunos problemas.	Interfaz deficiente o difícil de usar.	A: Hasta 10 pts. B: Hasta 5 pts C: 0	3
TOTAL				**100**	**61**